AI Workflow Orchestrator — Full Reference Implementation and Testing

Test cases to check backend implementation on SwaggerUI

1. Health check

Test

`GET /health`

Expected

{"status":"ok"}

---

2. Create a basic procurement request

Endpoint

`POST /requests`

Body

```
{
 "title": "Purchase laptop for new joiner",
 "description": "Need to buy a Dell laptop for a new employee joining next week.",
 "created_by": "Avantika",
 "request_type": "PROCUREMENT",
 "amount": 4500
}
```

Expected

- `id` is generated like `REQ-...`

- `state` = `DRAFT`
- `request_type` = `PROCUREMENT`
- `amount` = `4500`
- request appears in `GET /requests`

---

3. Create a leave request

Body

```
{
 "title": "Annual leave for family event",
 "description": "I need 2 days of annual leave next month for a family function.",
 "created_by": "Avantika",
 "request_type": "LEAVE",
 "leave_days": 2
}
```

Expected

- `state` = `DRAFT`
- `leave_days` = `2`

---

4. Create a support request

Body

```
{
 "title": "Production dashboard access issue",
 "description": "Unable to access the production dashboard since morning. This is
blocking reporting.",
 "created_by": "Avantika",
 "request_type": "SUPPORT",
 "severity": 4
}
```

Expected

- `request_type` = `SUPPORT`
- `severity` = `4`

---

5. List all requests

Endpoint

`GET /requests`

Expected

- returns all created requests
- includes the records created in earlier tests

---

6. Get one request by ID

Use an ID from a previous create response.

Endpoint

`GET /requests/{request_id}`

Expected

- exact request returned
- matching `id`

---

7. Update a draft request

Create a new draft first, then update it.

Endpoint

`PATCH /requests/{request_id}`

Body

```
{
 "title": "Purchase laptop and monitor for new joiner",
 "amount": 6200
}
```

Expected

- fields update successfully
- `updated_at` changes
- state remains `DRAFT`

---

8. Update should fail if request is no longer draft

First:

- create request
- submit it

Then try patching it.

Expected

- HTTP `400`
- message similar to:

{"detail":"Only DRAFT requests can be edited"}

---

9. Enrich an unknown request

Create request

```
{
 "title": "Need software license urgently",
 "description": "Please arrange purchase of a Figma enterprise license for design team.",
 "created_by": "Avantika",
 "request_type": "UNKNOWN"
```

}

Then call:

`POST /requests/{request_id}/enrich`

Expected

- AI or fallback classifies it
- likely `request_type` becomes `PROCUREMENT`
- `summary` gets populated
- `ai_confidence` gets populated

---

10. Enrich a leave-type unknown request

Create

```
{
 "title": "Need time off next week",
 "description": "I will be unavailable for 5 days due to personal travel.",
 "created_by": "Avantika",
 "request_type": "UNKNOWN"
}
```

Expected after enrich

- likely becomes `LEAVE`
- may infer leave-related summary

11. Submit procurement request below 10000

Create

```
{
 "title": "Buy office chair",
 "description": "Need an ergonomic chair for workstation.",
 "created_by": "Avantika",
 "request_type": "PROCUREMENT",
 "amount": 8000
```

}

Then submit.

Expected

- `state` = `IN_REVIEW`
- `approval_chain` = `["MANAGER", "FINANCE"]`
- `approval_index` = `0`
- `current_required_role` = `MANAGER`

---

12. Submit procurement request above 10000

Create

```
{
 "title": "Procure 5 new laptops",
 "description": "Need 5 laptops for new project team.",
 "created_by": "Avantika",
 "request_type": "PROCUREMENT",
 "amount": 55000
}
```

Expected after submit

- `approval_chain` = `["MANAGER", "FINANCE", "DIRECTOR"]`

---

13. Submit leave request of 2 days

Create

```
{
 "title": "Short leave request",
 "description": "Need 2 days leave for personal work.",
 "created_by": "Avantika",
 "request_type": "LEAVE",
 "leave_days": 2
```

}

Expected after submit

- `approval_chain` = `["MANAGER"]`

---

14. Submit leave request above 3 days

Create

```
{
 "title": "Extended leave request",
 "description": "Need 5 days leave for travel.",
 "created_by": "Avantika",
 "request_type": "LEAVE",
 "leave_days": 5
}
```

Expected after submit

- `approval_chain` = `["MANAGER", "HR"]`

---

15. Submit support request with low severity

Create

```
{
 "title": "VPN issue",
 "description": "VPN is not connecting on office laptop.",
 "created_by": "Avantika",
 "request_type": "SUPPORT",
 "severity": 2
}
```

Expected after submit

- `approval_chain` = `["SUPPORT_L2"]`

---

16. Submit support request with high severity

Create

```
{
 "title": "Critical outage in production",
 "description": "Production reporting is down for all users.",
 "created_by": "Avantika",
 "request_type": "SUPPORT",
 "severity": 5
}
```

Expected after submit

- `approval_chain` = `["SUPPORT_L2", "MANAGER"]`

---

17. Approve request with correct role

Take a submitted procurement/leave/support request where current role is `MANAGER`.

Endpoint

`POST /requests/{request_id}/approve`

Body

```
{
 "actor_name": "Rohit",
 "actor_role": "MANAGER",
 "comments": "Looks good"
}
```

Expected

- request moves to next approval step or `APPROVED`
- `approval_index` increments

---

18. Approve request with wrong role

Use a request waiting for `MANAGER`, but send:

```
{
 "actor_name": "FinanceUser",
 "actor_role": "FINANCE",
 "comments": "Approving"
}
```

Expected

- HTTP `403`
- error saying approval requires manager

---

19. Final approval path for high-value procurement

For request with chain:
`MANAGER -> FINANCE -> DIRECTOR`

Test approvals in sequence:

Step 1

`MANAGER` approves
Expected: still `IN_REVIEW`, next role `FINANCE`

Step 2

`FINANCE` approves
Expected: still `IN_REVIEW`, next role `DIRECTOR`

Step 3

`DIRECTOR` approves
Expected: `APPROVED`, `sla_deadline` becomes `null`

---

20. Reject request with correct role

Use an `IN_REVIEW` request.

Body

```
{
 "actor_name": "Rohit",
 "actor_role": "MANAGER",
 "comments": "Insufficient details"
}
```

Expected

- `state` = `REJECTED`

---

21. Reject request with wrong role

Use a request needing `MANAGER`, but reject with `HR`.

Expected

- HTTP `403`

---

22. Escalate request manually

Use a submitted or in-review request.

Endpoint

`POST /requests/{request_id}/escalate`

Body

```
{
 "actor_name": "AdminUser",
 "actor_role": "ADMIN",
 "reason": "Urgent business priority"
}
```

Expected

- `state` = `ESCALATED`

---

23. Escalate should fail for rejected request

Use a request already `REJECTED`.

Expected

- HTTP `400`
- only submitted/in-review requests can be escalated

---

24. Audit log should capture all actions

For one request, do this flow:

- create
- enrich
- submit
- approve

Then call:

```
GET /requests/{request_id}/audit
```

Expected

audit contains entries for:

- `CREATED`

- `AI_ENRICHED`
- `SUBMITTED`
- `APPROVED`

---

25. Request not found

Endpoint

`GET /requests/REQ-DOESNOTEXIST`

Expected

- HTTP `404`

---

26. Validation: missing title

Body

```
{
 "description": "Need laptop urgently",
 "created_by": "Avantika",
 "request_type": "PROCUREMENT"
}
```

Expected

- HTTP `422`

---

27. Validation: title too short

Body

```
{
 "title": "Hi",
```

"description": "Need laptop urgently",
"created_by": "Avantika",
"request_type": "PROCUREMENT"
}

Expected

- HTTP 422
  because title minimum length is 3

---

28. Validation: invalid severity

Body

{
"title": "Support issue",
"description": "System not working",
"created_by": "Avantika",
"request_type": "SUPPORT",
"severity": 9
}

Expected

- HTTP 422
  because severity must be between 1 and 5

---

29. Validation: negative amount

Body

{
"title": "Purchase request",
"description": "Need equipment",
"created_by": "Avantika",
"request_type": "PROCUREMENT",
"amount": -500

}

Expected

- HTTP 422

---

30. Persistence test

This is important because you moved away from in-memory store.

Steps

1. create a request
2. check `GET /requests`
3. stop server
4. restart server
5. check `GET /requests` again

Expected

- request should still exist in DB

This completes an overall validation test for the application using the backend software (SwaggerUI). From my experience, I have seen that all of these tests pass.

Implementation in Phases - Errors and Lessons

Phase 1: In-memory workflow engine for rapid prototyping

Used in built data structures to store audit logs and requests/actions.

1. LLM Output Schema Mismatch

Problem:
The LLM returned `"severity": "high"` instead of an integer, causing validation errors in FastAPI/Pydantic.

Solution:
Implemented a normalization layer in the workflow engine to:

- map `"low" → 1`, `"medium" → 3`, `"high" → 5`
- enforce strict integer validation before persistence

2. LLM Provider Integration (Groq)

Problem:
Initial integration failed due to:

- incorrect environment variable usage
- deprecated model IDs

Solution:

- Switched to Groq with OpenAI-compatible API
- updated model to `llama-3.3-70b-versatile`
- ensured environment variables were correctly loaded

3. Fallback Handling

Problem:
System needed to work even without API keys.

Solution:
Implemented a rule-based fallback system:

- keyword-based classification
- default summary generation
- fixed confidence score (0.45)

4. LLM Reliability

Problem:
LLM outputs are non-deterministic and may break strict schemas.

Solution:

- added post-processing validation layer
- enforced type safety before updating request objects
- ensured system remains deterministic

5. Separation of Concerns

Design Decision:

- LLM used only for **enrichment (classification, summary)**
- Workflow logic remains **rule-based and deterministic**

Reason:
Ensures:

- reliability
- auditability
- enterprise readiness

Phase 2: Database-backed architecture using SQLAlchemy

There were some errors that came up while testing because the architecture was changed with the addition of an SQLite database. I used SQLAlchemy for the same.

1. The app was still using the in-memory store instead of the database

What happened

Even though I had `db.py`, `orm_models.py`, and `repository.py`, the actual API flow was still using in program storage inside `engine.py`.

So the project looked database-ready, but in reality:

- data was still being saved in Python dictionaries
- nothing was being written to SQLite
- restarting the server would erase everything

How I fixed it

I updated `engine.py` so it stopped using `store` and instead used:

- `WorkflowRepository`
- `AsyncSession`
- repository methods like `create_request()`, `get_request()`, `list_requests()`

Lesson

This error taught me that **having DB files is not enough**. The business logic layer must actually call the DB layer.

2. `main.py` was still written for the old synchronous engine pattern

What happened

The old `main.py` imported a global engine object and called methods directly.

That worked in the memory-based version, but once the database layer became async, the route handlers also needed to become async and needed access to a DB session.

How I fixed it

I changed:

- route handlers to `async def`
- injected `db: AsyncSession = Depends(get_db_session)`
- created `WorkflowEngine(db)` inside each route
- added startup table creation using FastAPI lifespan

3. `WorkflowEngine() takes no arguments`

What happened

After updating `main.py`, the app crashed with:

TypeError: WorkflowEngine() takes no arguments

How I fixed it

I added:

```
def __init__(self, session: AsyncSession) -> None:

  self.repo = WorkflowRepository(session)
```

to `WorkflowEngine`.

I also removed the old pattern:

```
engine = WorkflowEngine()
```

from the bottom of the file.

4. Missing update method in the repository layer

What happened

The repository had methods for:

- create
- get
- list
- add audit event
- get audit events

But the workflow also needs to update requests many times after creation:

- when enriching
- when submitting
- when approving
- when rejecting
- when escalating

The repository did not yet have a method to update an existing request row.

How I fixed it

I added:

```
async def update_request_record(...)
```

to the repository.

That method:

- looked up the request by ID
- updated each field
- committed the transaction

So I stayed loyal to the **CRUD ARCHITECTURE.**

5. `Random Indentation errors`

6. Approval chain looked missing in created requests

What happened

When I created a request and looked at it immediately, `approval_chain` was empty, which looked like a bug.

Example:

"state": "DRAFT",

"approval_chain": [],

"current_required_role": null

This was because the approval chain is **not assigned during creation**.

It is only built during:

submit_request()

So while the request is still in `DRAFT`, the empty chain is expected behavior.

Phase 3 and 4: Deployment Challenges

1. `pydantic-core` build failed on deployment

The first big issue was that Streamlit/Render was trying to install dependencies with **Python 3.14**, and `pydantic-core`failed during build.

What this taught:

- hosted platforms may use a newer Python version than your local machine
- dependency compatibility matters during deployment, not just in local dev

2. Accidental Streamlit UI bug showing `st.image` docstring

At one point, the app displayed the documentation for `st.image(...)` instead of an image. That happened because the function was being referenced instead of called.

What this taught us:

- not every “deployment issue” is really deployment
- sometimes the app is running, but there is a frontend code bug

3. Streamlit could not connect to `127.0.0.1:8000`

The deployed Streamlit app showed a connection refused error when trying to call:

http://127.0.0.1:8000/requests

That happened because `localhost` worked on your laptop, where FastAPI and Streamlit were both running locally, but not in production.

What this taught me:

- local architecture does not automatically work in cloud deployment
- once frontend and backend are deployed separately, Streamlit must call the public backend URL, not localhost

4. Render root URL showed `{"detail":"Not Found"}`

When I opened the Render URL directly, you saw “Not Found.”

That looked scary at first, but it actually meant the FastAPI backend was live and just didn’t have a `/` route.

What this taught me:

- `Not Found` at `/` can be normal for FastAPI
- always test proper endpoints like `/docs` or `/health`

5. Backend business logic error: `Could not determine approval chain`

After Streamlit and Render were connected, the next error came from the workflow logic itself.

That meant deployment was actually successful, but the backend could not decide the approval chain for a request.

What this taught us:

- once infra is fixed, business-logic errors become visible
- deployment debugging often uncovers application-level bugs that were hidden before

6. LLM was not being used

The app kept saying:

Using fallback enrichment because client/model is missing

That happened because the environment variables for the LLM provider were not set on Render.

What this taught us:

- local `.env` values do not automatically exist in cloud deployment
- hosted apps need environment variables to be manually configured

7. Database disappeared after deployment/restart

Later, the app worked and the LLM worked, but the requests vanished.

That happened because the backend was using a local SQLite database file on Render's ephemeral filesystem.

What this taught me:

- cloud services may not preserve local files between restarts/redeploys
- production apps need persistent storage, like Postgres or a mounted persistent disk

---

Main lessons learned

1. Deployment is not just “push and run”

You have to think about:

- Python version
- dependency compatibility
- environment variables
- database persistence
- frontend-backend connection

2. Localhost is only for local development

This was one of the biggest lessons.

Locally:

- Streamlit can call `127.0.0.1:8000`

In production:

- Streamlit must call the deployed FastAPI URL on Render

3. Separate services need clear configuration

Once frontend and backend are on different platforms, you need:

- a proper `API_BASE_URL`
- correct secrets/env vars
- CORS awareness if needed

4. Environment variables are critical

The LLM issue was not a model problem at all. It was config.

So now I learned:

- API keys
- provider names
- model names
  must all be set correctly in the deployment platform

5. Cloud databases need persistence

A local SQLite file is okay for testing, but not ideal for a deployed app unless it sits on persistent storage.

Best lesson here:

- for a proper deployed full-stack project, use Postgres

6. Logs are your best friend

Almost every issue got solved by reading the logs carefully:

- build logs showed Python/dependency problems
- runtime logs showed fallback LLM mode
- frontend errors showed localhost connection problems

7. A deployment can be “successful” even if the app still has logic issues

You reached a point where:

- backend was deployed
- frontend was deployed
- LLM was working

and only then did the approval-chain/database issues show up.

Phase 3: Streamlit frontend addition

Streamlit Components planned:

1. **Create Request page**

- title
- description
- request type
- amount / leave days / severity fields
- submit button

2. **Approval Dashboard**

- request state
- current required role
- approval chain
- approval index
- approve / reject buttons

3. **All Requests page**

- table of all requests
- filters by state/type
- select a request to inspect

4. **Analytics / Summary page**

- counts by state
- request type distribution
- pending approvals

1. Request was created but not submitted

At first, the Streamlit create form only called the **create request** endpoint, so the request stayed in `DRAFT`.

Effect

- approval chain was empty
- `current_required_role` stayed null
- request did not move into workflow

Fix

After creating the request, the frontend also had to call:

/requests/submit

so the backend could generate the approval chain.

---

2. `405 Method Not Allowed`

When trying to submit requests from Streamlit, you got:

API Error 405: {"detail":"Method Not Allowed"}

Cause

The frontend was calling the correct-looking endpoint, but with the **wrong method or wrong route shape**.

Fix

You checked Swagger and aligned the Streamlit API call with the actual backend endpoint definition.

---

3. `422 Field required` on approve/reject

During approval actions, you got errors like:

API Error 422: {"detail":[{"type":"missing","loc":["body"],"msg":"Field required"...}]}

and later:

"actor_name" field required

"actor_role" field required

Cause

The frontend request body did not match what the FastAPI backend expected.

Fix

You updated the payload to send the correct fields, such as:

- `request_id`
- `actor_name`
- `actor_role`

instead of older/mismatched names like `approver_role`.

---

4. Approval dashboard existed, but not as a real dashboard

Initially, the frontend had request details and action buttons, but not a proper dashboard view.

Missing pieces

- top summary metrics
- filtered approval queue
- pending approvals section
- approval progress display

Fix

You added a dedicated **Approval Dashboard** page showing queue data, state counts, and action controls.

---

5. Frontend not reflecting backend changes

After updating backend categories like `FINANCE`, Swagger worked, but Streamlit did not immediately reflect it.

Cause

Possible causes included:

- Streamlit still running old code
- frontend dropdowns still hardcoded with old categories
- app not restarted properly

Fix

You updated the Streamlit page files and restarted the app fully.

---

6. `ModuleNotFoundError: No module named 'app'`

You saw this when trying to run files like:

python app/engine.py

python app/ai_llm.py

Cause

Those files use imports like:

from app.models import ...

which only work when the project is run from the root as a package.

Fix

Instead of running individual files directly, you ran:

- backend with `uvicorn app.main:app --reload`
- frontend with `streamlit run frontend/streamlit_app.py`

---

7. Invalid datetime import in backend

You got:

ImportError: cannot import name 'utc_now' from 'datetime'

Cause

`utc_now` is not part of Python's `datetime` module.

Fix

You replaced it with:

- `datetime.utcnow()` or
- better, `datetime.now(timezone.utc)`

---

8. Audit logs were not visible in frontend

You realized audit logs were stored in a **separate database**, so they were not automatically shown in request details.

Cause

The request API returned request data only, not audit-log data.

Fix

You identified that audit logs need either:

- a separate endpoint, or
- explicit retrieval and display in Streamlit

---

9. Streamlit theme config not applying

When you added `.streamlit/config.toml`, you initially did not see UI changes.

Cause

Usually due to:

- wrong folder location
- not restarting Streamlit
- running the app from the wrong directory

Fix

You placed the file in the project root under:

.streamlit/config.toml

and restarted Streamlit properly.

---

10. Category design felt too limited for LLM classification

You noticed that with only:

- `LEAVE`
- `PROCUREMENT`
- `SUPPORT`

the LLM was forced to squeeze many real-world requests into weak categories.

Example

A theft/security issue had no natural category.

Fix / insight

You decided the approval chains were fine, but the classification space might need expansion, such as adding:

- `FINANCE`
- `HR`

Here's a clean summary of the **deployment errors** and **lessons learned** from your AI Workflow Orchestrator deployment.

Deployment errors we faced

1. `pydantic-core` build failed on deployment

The first big issue was that Streamlit/Render was trying to install dependencies with **Python 3.14**, and `pydantic-core`failed during build.

What this taught us:

- hosted platforms may use a newer Python version than your local machine
- dependency compatibility matters during deployment, not just in local dev

2. Accidental Streamlit UI bug showing `st.image` docstring

At one point, your app displayed the documentation for `st.image(...)` instead of an image. That happened because the function was being referenced instead of called.

What this taught us:

- not every "deployment issue" is really deployment
- sometimes the app is running, but there is a frontend code bug

3. Streamlit could not connect to `127.0.0.1:8000`

Your deployed Streamlit app showed a connection refused error when trying to call:

http://127.0.0.1:8000/requests

That happened because `localhost` worked on your laptop, where FastAPI and Streamlit were both running locally, but not in production.

What this taught us:

- local architecture does not automatically work in cloud deployment
- once frontend and backend are deployed separately, Streamlit must call the public backend URL, not localhost

4. Render root URL showed `{"detail":"Not Found"}`

When you opened the Render URL directly, you saw "Not Found."

That looked scary at first, but it actually meant the FastAPI backend was live and just didn't have a `/` route.

What this taught us:

- `Not Found` at `/` can be normal for FastAPI
- always test proper endpoints like `/docs` or `/health`

5. Backend business logic error: `Could not determine approval chain`

After Streamlit and Render were connected, the next error came from the workflow logic itself.

That meant deployment was actually successful, but the backend could not decide the approval chain for a request.

What this taught us:

- once infra is fixed, business-logic errors become visible
- deployment debugging often uncovers application-level bugs that were hidden before

6. LLM was not being used

The app kept saying:

Using fallback enrichment because client/model is missing

That happened because the environment variables for the LLM provider were not set on Render.

What this taught us:

- local `.env` values do not automatically exist in cloud deployment
- hosted apps need environment variables to be manually configured

7. Database disappeared after deployment/restart

Later, the app worked and the LLM worked, but the requests vanished.

That happened because the backend was using a local SQLite database file on Render's ephemeral filesystem.

What this taught us:

- cloud services may not preserve local files between restarts/redeploys
- production apps need persistent storage, like Postgres or a mounted persistent disk

---

Main lessons learned

1. Deployment is not just “push and run”

You have to think about:

- Python version
- dependency compatibility
- environment variables
- database persistence
- frontend-backend connection

2. Localhost is only for local development

3. Separate services need clear configuration

4. Environment variables are critical

5. Cloud databases need persistence

6. Logs are your best friend

7. A deployment can be “successful” even if the app still has logic issues

---